Faustus

Faustus records and inspects Path of Exile economy data. The Rust workspace contains poe.ninja economy collection, historical dump imports, SQLite storage, and a market service. The Aurelia v2/TypeScript frontend runs in a browser or a Tauri v2 desktop shell. The previous GPUI implementation and legacy GGG integration are preserved in `crates/market-ui-deprecated` and the Rust crates, but normal service operation no longer collects GGG exchange history.

Run Locally

Prerequisites: Rust 1.92+, Node.js 22.12+ (or a newer supported Node release), npm, and the Tauri Windows prerequisites (MSVC build tools and WebView2). Aurelia v2 currently uses the pinned `2.0.0-rc.2` release.

Install frontend dependencies once:

```
Push-Location crates\market-ui
npm ci
Pop-Location
```

Start the backend and Windows Tauri app together:

```
.\scripts\dev.ps1
```

The development script builds and starts the loopback backend, waits for its health endpoint, then launches Tauri/Vite in the foreground. Closing the desktop app stops the development backend. Service logs are written under `$env:TEMP\Faustus`. Use `.\scripts\dev.ps1 -Web` to start the same service with the browser frontend at `http://127.0.0.1:5173`, or `-Deprecated` to launch GPUI instead.

The client loads the poe.ninja economy leagues, uses the service's valid selection or defaults to the first entry (the current temporary softcore trade league), and tells the service which league to watch. The service validates that selection and starts collecting immediately. Changing the dropdown switches the collector to the newly selected league. League options are cached by the service for 15 minutes.

To run the backend separately:

```
cargo run -p market-service -- serve
```

To trigger a single collection manually instead:

```
cargo run -p market-service -- ingest-once --league "LEAGUE_ID"
```

Each `ingest-once` run collects supported poe.ninja economy categories, using the same provider cache and validators as the scheduled collector. It does not collect GGG exchange pages.

When running the backend separately, launch the desktop frontend in another terminal:

```
cd crates\market-ui
npm run tauri -- dev
```

Use `npm run dev` instead for the browser frontend. Vite proxies `/api` and `/health` to `http://127.0.0.1:8787`; both clients use ordinary HTTP `fetch`, not Tauri commands or an HTTP plugin. Development builds use the external service. Packaged release builds run the same backend inside the desktop process and stop it when the application exits; no separate service executable or setup is needed.

Distribute to Friends (Windows)

From `crates\market-ui`, run `npm run tauri -- build` after installing the development prerequisites above. Send the generated NSIS setup executable from `target\release\bundle\nsis` (or the corresponding directory under your configured Cargo target directory). Your friends do not need Rust, Node.js, npm, or a separately running backend. The installer uses Tauri's default WebView2 bootstrapper when the runtime is missing; first-time installation may need internet access.

The installer contains the application, frontend assets, and database schema, **not your local database, downloaded archives, or saved prices**. The first launch creates an empty database at `%APPDATA%\com.faustus.market\faustus.sqlite3`. Each Windows user has their own data, retained across app restarts and upgrades. Packaged builds deliberately do not use the development working-directory database or `FAUSTUS_DATABASE` / `FAUSTUS_ECONOMY_DATABASE`.

- **Live league:** the app selects the current available league on launch, fetches immediately, and collects economy/gem snapshots every 15 minutes while open. Changing the live selection switches collection. Browsing historical data keeps the last live league collecting. Collection stops when the app closes; it is not a Windows background service.
- **Historical leagues:** open **Settings** → **Historical datasets**, then click **Import** only for the leagues you want. This downloads the published poe.ninja archive and imports it locally, with status/progress and retry errors shown there. **Discover dumps** fetches only the catalog, not league histories. Completed datasets become available in **League / dataset** and remain saved for future launches.
- **First run:** internet access is required for the live league catalog/prices and optional archive downloads. No historical leagues are automatically imported. Downloaded archives can consume substantially more disk space after import.
- **Local backend:** release builds listen only on `127.0.0.1:8787`. Close other Faustus instances or development services before launching; a port conflict or database startup failure shows an error dialog rather than connecting to another process's data.

Do not distribute the repository folder or your `faustus.sqlite3` files as the application package. Historical downloads retain their published poe.ninja CC BY-SA attribution.

GitHub Releases and Installer Updates

The **Release Windows installer** GitHub Actions workflow is **manual only**, and runs from `main`. It reads the application version from `crates\market-ui\package.json`; Tauri reads that same file directly. Rust crate versions are internal package versions, not the installer/update version. Use stable versions such as `0.1.1`, without a `v` prefix or prerelease suffix.

One-time setup (keep the same signing key for all future releases):

1. Make this repository and its release assets public. Private GitHub assets require authentication and are not supported by this updater; do not embed a GitHub token in the app.
2. Generate a password-protected updater key **outside the repository**, from `crates\market-ui`:

   ```
   New-Item -ItemType Directory -Force "$env:USERPROFILE\.tauri" | Out-Null
   npm run tauri -- signer generate -w "$env:USERPROFILE\.tauri\faustus.key"
   ```

   Follow the password prompts. Back up the private key and password securely; losing them prevents updates to already-installed releases. Never commit or share the private key.
3. In the repository's **Settings** → **Secrets and variables** → **Actions**, add:
   - **Secret** `TAURI_SIGNING_PRIVATE_KEY`: the full contents of the generated `faustus.key`.
   - **Secret** `TAURI_SIGNING_PRIVATE_KEY_PASSWORD`: its password.
   - **Variable** `TAURI_UPDATER_PUBLIC_KEY`: the full contents of `faustus.key.pub` (the public key, not its path).
4. Ensure GitHub Actions can create releases using the workflow's `contents: write` permission.

For each release:

```
cd crates\market-ui
npm version patch --no-git-tag-version   # Or: npm version 0.2.0 --no-git-tag-version
```

Commit and push the updated `package.json` **and** `package-lock.json`, along with the application changes. Then open **Actions** → **Release Windows installer** → **Run workflow** → **main**. Do not create the tag yourself: the workflow creates `v<version>` from the selected commit.

The workflow tests the app, builds the Windows x64 NSIS installer, signs the update artifact, and uploads the installer, its `.sig`, and `latest.json` to a draft release. It publishes and marks that release latest only after the required assets are uploaded. An already-published version is skipped, never overwritten; a version older than or equal to latest is rejected. An orphaned tag or unfinished draft fails explicitly: inspect and delete that draft/tag before retrying the same version, or bump the version. Failed builds before release creation can simply be retried.

GitHub-built releases check `https://github.com/Souchy/Faustus/releases/latest/download/latest.json` on launch. Newer versions prompt **Update now / Later**; no installer downloads or runs without confirmation. **Help → Check for updates...** retries manually and reports up-to-date/network errors. Downloads must pass Tauri's signature verification before installation. Windows shows installation progress and restarts the app; the per-user database is retained. An interrupted historical import may need Retry afterward.

Ordinary local builds leave the public key empty and do not generate signed update artifacts or check on launch; the manual check explains that updates are unconfigured. Install the first GitHub-built release manually to bootstrap updating, including when replacing an older installer without updater support. Update signing is separate from Windows Authenticode signing: these installers may still show SmartScreen warnings.

The **Current prices** tab is an economy workspace with grouped General, Items, Uniques, Atlas, and Crafting navigation. It covers the supported PoE 1 economy categories, with Exchange/Stash source switching where the public API supports both. Search is category-local and independent of Gem flipping. Tables support sorting, 100-row batches, native/Chaos/Divine quotes, variant details, recent trend charts, and category-specific filters for item type, gem level/quality, links, corruption, map tier, and listing counts where supplied. Clear filters appears only when category-specific filter controls are available, not for search and quote alone. The category navigation, heading, filters, item list and item details scroll together within the Current prices page; the shared app header remains accessible. Empty, failed, and unsupported source combinations are explicit. Prices are aggregate estimates, not executable buy/sell orders.

Items contains gems, cluster jewels, Wombgifts, Corpses and Incubators. Uniques groups unique equipment, including Forbidden Jewels and Shrine Belts; Unique Maps remain in Atlas. These navigation groups do not change category identifiers, price sources or stored history.

Click an item to see its available properties/modifiers and local price history. Charts retain native units, separate daily dumps from timestamped observations, and break at missing intervals rather than inventing prices. A history data table provides the same observations without relying on the chart. Range filters are relative to that item's newest saved observation, so they also work for old leagues. Historical quotes use same-day currency rates when available, never today's rates; unavailable conversions display a dash. The top-level **League / dataset** selector groups Live and Historical options and applies to both Current prices and Gem flipping. **As of**, beside it in the header, selects the historical day shared by the tabs. Available days follow the active price category or historical gems; live Gem flipping shows its saved snapshot instants there instead. A league can have both live feeds and imported history. Choosing a live league changes the collector selection as before; browsing an archive never selects that archive for live collection.

Historical Gem flipping uses imported SkillGem variants and same-day Chaos-denominated Currency prices for GCP and Vaal costs. Level-only dump variants such as `20` mean level 20, zero quality; `21c` also marks corruption. Existing imports are decoded when read, without requiring reimport. Level, quality and approximate Vaal-corruption strategies use the same analysis engine as live data, but results are daily price comparisons, not executable trades or a simulation of historical game mechanics. Missing gem prices or material costs are never filled with current data. Manual material overrides remain explicit assumptions, including optional Lens costs. Archive listing/sample counts, confidence, liquidity and precise gem-type metadata are unknown: listings show `N/A`, type shows `Unknown`, and confidence controls are hidden rather

than filtering every archived row out. Switching back to a live league restores the existing saved-instant and confidence controls.

Historical table trends use the eight daily samples spanning the seven days ending on the selected date, never later prices. Missing days remain gaps; a seven-day percentage requires valid, nonzero baseline and end observations in the same currency. Icons are display-only enrichment from saved poe.ninja supported overview metadata (fetched from the Standard overview when no metadata is cached), served by Grinding Gear Games' `web.poecdn.com` image CDN. Metadata prefers Standard and Stash per item name, but retains Exchange-only items. The dump's broad Currency group also checks Fragment and other exchange-category metadata. Missing currency/fragment icons additionally check the full Standard Currency and Fragment `currencyDetails` catalogs through the shared overview cache, including entries without live prices (such as Unrelenting Timeless emblems). These icon-only entries never become price observations. Exact names are matched using compatible saved provider IDs or normalized variants (including map generations and influence variants); unambiguous artwork is a fallback. Missing currency/fragment icons without variants (including historical reliquary keys) also use exact, unambiguous names from GGG's official trade static metadata, cached in memory for 24 hours. Both sources accept only HTTPS images from `web.poecdn.com`. Missing or genuinely ambiguous matches stay blank; metadata-fetch errors are shown in the workspace. This never joins price identities, substitutes current prices, or supplies historical listing counts.

In item details, drag anywhere on the compact overview graph to select a custom period. The drag endpoints snap to saved observations, are sorted regardless of drag direction, and clamp to the full history's left/right endpoints even when released outside the graph. The start/end handles remain individually draggable, with arrow/Home/End keys available when focused. Double-click either handle (or press Enter while focused) to reset to all available history; presets also adjust the range without refetching. Period labels show dates only, while snapshot hover labels retain hours/minutes (UTC) without subsecond precision. The price chart has observation dots, adaptive price-axis increments, day labels, and subtle gridlines. Hover it to show a vertical guide and the nearest saved observation's date/time, price, currency, and granularity; missing prices are not interpolated. A thin volume chart below it shows provider-reported quote-currency volume at each saved snapshot, not summed daily trades. Volume is retained for newly collected observations; historical dumps and older snapshots have no recorded volume and display an explicit unavailable state instead of fabricated zeros. Price axes, volume, and the history table all follow the selected period. History rows scroll with the item detail page rather than inside a compressed nested pane.

Custom graph periods retain their selected dates while browsing other items, categories, sources, live leagues, or archived datasets in the current workspace session. Each history snaps those dates to its nearest available observations, clamping to an endpoint if necessary, without overwriting the original selection. Returning to an item with full coverage restores the original period, even after empty or failed history requests. Changing a handle or dragging a new range replaces the selection; reset clears it. Trailing-day presets retain their selected duration and remain relative to each item's newest observation.

Economy numbers use spaces as thousands separators and a decimal comma (for example, `1 234 567,89`), including table values, item details, chart axes/tooltips, volume, and observation counts. Stored and API numeric values retain their original machine-readable format.

Item details replace the category title with a `Category > Item` breadcrumb. Click the category in that breadcrumb to return to the list with its filters and selected graph period preserved.

Current prices remains poe.ninja-only. The separate **Currency Exchange** page (`#/exchange`) browses GGG's PC hourly markets, **Flip analysis** (`#/flips`) compares historical growth across categories, and **Triangular arbitrage** (`#/triangular-arbitrage`, with `#/arbitrage` retained as an alias) finds implied historical three-market loops. Existing legacy GGG records and endpoints remain separate from the compact hourly store.

GGG hourly exchange history

On Currency Exchange, choose a league and an inclusive start/end range, then use **Fetch missing hours**. Imports are explicit: opening a page, selecting a league or loading analysis does not start a full-league download. All completed hours can be retained; there is no sampling or automatic daily compaction. Select a league from the shared selector or enter a retained GGG league on the Exchange page. Requests are limited to 180 days per job, with dates entered/displayed in local time and converted to aligned UTC hours. The current unfinished hour cannot be imported.

The backend fetches one GGG response per missing hour and saves all native market pairs for the selected league, including pairs involving neither Chaos nor Divine Orbs. Long item and market identities are interned in catalogs; all traded volumes, stock bounds and ratio bounds are stored as integer columns, without permanent all-league raw JSON copies. Names and categories use exact metadata IDs from the maintained RePoE dataset, with conservative known-currency fallbacks; this is metadata enrichment, never a source of prices or price-identity substitutions. Unknown items retain their exact GGG IDs and partial metadata coverage is reported explicitly.

Coverage is stored independently of market rows. A valid hourly response and its selected-league markets are committed together; an hour with no trades is still fetched, but zero traded amounts never become a zero price. Failed and missing hours remain distinguishable. Repeating an overlapping import skips completed hours. Imports can be cancelled, and interrupted imports can be resumed from persisted coverage after a service restart. Cancelling or navigating away does not delete observations. No old global cursor collector is re-enabled.

The market browser displays native units and historical traded amounts. Its pair history breaks at missing hours and distinguishes unfetched hours from successfully imported hours without trades. GGG publishes hourly aggregates, not individual trades, live directional quotes or a complete order book. Hourly traded price is quote volume divided by item volume; reversing that average does not establish an executable opposite-side quote.

Historical growth and arbitrage

Flip analysis selects two completed hourly endpoints and minimum/maximum **starting** prices in Chaos. It returns one row per item/variant across all categories, with start/end prices, absolute growth, percentage growth, provider and Exchange/Trade venue. Percentage growth is `(end / start - 1) * 100`; zero or unavailable starting prices are unrankable. The server filters and globally ranks endpoint deltas before returning a page of at most 100 rows, with a matching total and stable item tie-breaks. Search, category, venue and growth sorting apply to the full result, not just the visible page.

Long item names wrap within a bounded button on desktop and mobile, while numeric columns stay on one line.

Expanded flip native-history observations show ratios at least 1 by reversing sub-unit ratios and their item labels: `0.5 The Chosen per Chaos Orb` becomes `2 Chaos Orb per The Chosen`, while `90 Wild Crystallised Lifeforce per Chaos Orb` remains unchanged. Missing/zero-trade prices remain missing. This is display-only; charts retain a fixed native B-per-A direction and stored observations and growth calculations are unchanged.

Strict start date is unchecked by default. Checking it reruns analysis using only the selected start hour (or UTC dataset day for archives), excluding later-starting items. Unchecked retains first-available in-period starts. The selected end remains strict in both modes. The choice persists when navigating between pages and applies to server-side filtering/ranking and pagination (`strict_start=true` on the flips API).

Cluster jewel labels include their poe.ninja item-level bracket (1-49, 50-67, 68-74, 75-83 or 84-100). CSV dumps omit this field; archive rows recover it only from an exact provider ID, name, passive variant and base-type match in saved metadata before filtering and ranking. Different item-level brackets remain separate even with identical names and passive counts. Missing or ambiguous matches show `item level unknown` and retain their distinct provider identities; historical prices are never replaced with current prices.

The selected start is the earliest permitted observation, not a requirement that every item already exists then. Items/pairs without a positive starting price use their first saved positive price inside the period, strictly before the ending observation. The selected end still requires an exact end-hour price (or UTC dataset day for archives). Actual starting dates are visible beside starting prices, and expanded alternatives show their own dates. Starts never move outside the period, across feeds/archives or to satisfy a minimum-volume threshold. GGG zero-trade hours are not prices. Missing imports can hide earlier activity; these are first **saved** observations, not guaranteed first-ever trades. Chaos conversion uses each actual observation's contemporaneous rate, not the selected start's rate.

Analyze reloads the known datetime bounds after imports and clamps valid entered dates independently, preserving the other endpoint and filters. Invalid date syntax still reports an error. When official league datetimes are unavailable, fully fetched empty outer GGG hours can tighten day-only archive bounds to the observed active exchange window; a partial import cannot shrink the league to its fetched subset. Empty exchange hours are not observation ticks. These bounds are explicitly labeled observed, not official. Items starting later remain eligible; items without a selected-end price remain excluded.

Each ranked row displays starting-hour → ending-hour item-unit volume for its selected GGG pair, not a period total or quote-currency volume. poe.ninja/archive rows show **Unknown** rather than a fabricated zero.

poe.ninja dump rows also show the provider's confidence at the start and end observations (`High`, `Medium`, `Low`, or **Unknown**); this is not traded volume or a confidence score invented by Faustus. GGG confidence is not applicable. Confidence is saved per observation in a compact side table, rather than taken from an item's latest metadata.

Minimum poe.ninja confidence mirrors the GGG minimum-volume filter: it applies only to poe.ninja/archive rows, leaving GGG rows unaffected. Either the actual starting **or** ending observation can

meet the chosen confidence/volume threshold. `Low or higher`, `Medium or higher`, and `High only` accept a row when at least one observation qualifies; Unknown cannot qualify by itself, but does not disqualify a known qualifying opposite endpoint. The default **All confidence levels** includes Unknown. These filters use endpoints, not period averages; confidence labels are not averaged numerically. Filtering happens globally before category totals, ranking and pagination, including when items start later than the selected period start. The choice persists across page navigation; the flips API accepts `min_confidence=Low|Medium|High`.

Global confidence filtering uses bounded batches and a covering archive/date confidence index, retaining the stored-price guard without random price-history reads. The index is built automatically on the first service startup after upgrading; existing prices and confidence values are unchanged.

After the ranking appears, a separate cancellable batch loads historical Chaos sparklines for only its displayed rows (at most 100). Each series has at most 60 evenly spaced sampled hours or UTC dataset days. Missing sampled prices/conversions break the line; unsampled fluctuations and gaps are not represented. A single observed point is a dot, not an invented trend. GGG keeps the row's selected native pair throughout and converts using each sampled hour's direct Chaos cross-rate; poe.ninja uses the same feed/archive/granularity and contemporaneous rates. Charts never delay the ranking or trigger provider-history downloads. Changing pages, filters or leagues cancels obsolete chart requests; failures remain visible without discarding the ranking.

Older imports did not retain the CSV `Confidence` column. Restore it without changing prices or rerunning a full import:

```
market-service --database D:\Faustus\faustus.sqlite3 backfill-confidence --name
Allflame --file G:\Downloads\Allflame
```

`--file` accepts either a dump ZIP or a directory containing the exact `<archive>.currency.csv` and `<archive>.items.csv` files. Only observations matching the completed import's league, identity, date, currency, archive and existing price are enriched. Unknown/mismatched values are not guessed; skipped CSV rows are reported. Future imports retain confidence automatically.

The small **i** disclosure beside Flip analysis contains the calculation caveats, timeline instructions, coverage notes and volume-filter explanation. It opens by click or keyboard and closes with Escape; errors, loading and unavailable-analysis states remain visible outside it. The start/end controls display local dates as `yyyy-mm-dd hh:mm` independently of browser locale, retain bounded calendar pickers, and replace a duplicate selected-period caption.

The period band shows actual saved hourly/daily observations. Drag either endpoint or the selected band, zoom around the pointer with the wheel, pan the visible window, or reset the view. Date inputs and handles stay inside the selected league's known bounds; defaults use available observations rather than an arbitrary recent window. When exact official league dates are unavailable, the UI explicitly identifies bounds derived from published archive dates or saved coverage. Missing data is never represented as a price observation. Period metadata uses a persisted observation-time manifest; endpoint reads use covering price and compact item-metadata indexes rather than loading complete histories or raw provider item payloads.

GGG is preferred for overlapping unvariant items when usable endpoint data exists. Cross-provider display deduplication uses unambiguous exact item names without variants; native price identities and histories are never joined by name. Unknown metadata stays separate rather than guessing an identity. The same native pair must have trades at both endpoints: direct Chaos is preferred if it meets the minimum item-volume filter, otherwise a qualifying pair is chosen by its minimum endpoint item volume, not by its largest apparent growth. Expandable alternatives retain native prices, Chaos conversions and endpoint volumes. Conversions use each endpoint's directly traded Chaos cross-rate; no current rates, arbitrary multi-hop routes or different buy/sell providers are substituted. Unsupported conversions are excluded explicitly.

poe.ninja comparisons retain the same feed/archive, quote currency and granularity. Existing start-hour snapshots use the latest saved observation in that hour; missing starts seek the first positive saved observation within the period. Daily archives use UTC dataset days and do not acquire hourly precision merely because an hour was selected. Item variants stay separate and actual observation dates remain visible. Later-starting items have shorter comparison windows; growth is not annualized or duration-normalized. The minimum GGG item-volume filter applies only to GGG observations: poe.ninja/archive rows remain included with explicitly unknown liquidity, not a fabricated volume guarantee. Imported dump provenance is labelled as a dump rather than claiming that every exported category was necessarily stash-derived. This page does not backfill the undocumented poe.ninja chart endpoint.

Triangular arbitrage evaluates Chaos-rooted three-market cycles within one completed GGG hour, retaining native pair rates. Returns are **implied historical discrepancies**, not executable real-time profit: the legs may have traded at different moments, and bid/ask spreads, available orders, gold costs, rounding and slippage are unknown. Its volume metric is the Chaos-equivalent bottleneck of historical traded amounts, not available trading capacity.

Historical economy dumps

Open **Settings** using the gear beside **Refresh** in the header to discover published poe.ninja economy dumps, import or retry individual archives, see import counts/date coverage/errors, and browse completed leagues. **Browse** returns to Current prices with that archived dataset selected. Import status polling stops when leaving Settings; imports continue in the service. In Settings, the header **Refresh** button refreshes the dataset catalog and import statuses rather than market prices. Repeated provider catalog entries are shown once per archive name, with the local import status taking precedence. **Discover dumps** checks for newly published archives, including Allflame once it appears; unpublished archives are not guessed or downloaded automatically.

Imports include only the named softcore challenge league. Hardcore, Ruthless, and permanent-league CSVs in the same ZIP are not imported. The initial selection is **Ancestors, Mirage, Phrecia 2.0, Keepers, and Mercenaries**. Imports are explicit, bounded, and idempotent. Incomplete jobs are hidden from historical queries and can be retried after failure/restart; a retry of the same checksum reuses existing observations without duplication. The importer streams semicolon-separated CSVs and preserves item IDs, base types, variants and link groups rather than joining prices by name. Archive IDs remain separate from live provider IDs when a reliable variant match is unavailable.

Per-archive OS locks beside the economy database prevent overlapping CLI/service imports. Locks are released automatically when a process exits; service restart recovery does not mark a still-running CLI job

as failed.

CLI equivalents:

```
cargo run -p market-service -- import --name "Ancestors"
cargo run -p market-service -- import --name "Ancestors" --file
"C:\data\Ancestors.zip"
# Explicitly replace a completed archive when reprocessing it is needed:
cargo run -p market-service -- import --name "Ancestors" --replace
```

Use one database for live economy updates, historical dumps, and saved Gem flipping snapshots. `--database` selects it for every command; `FAUSTUS_DATABASE` provides a shared environment default. Without either, the default is `faustus.sqlite3` in the working directory.

```
cargo run -p market-service -- --database "D:\Faustus\faustus.sqlite3" serve
cargo run -p market-service -- --database "D:\Faustus\faustus.sqlite3" import -
-name "Mirage"
.\scripts\dev.ps1 -Web -Database "D:\Faustus\faustus.sqlite3"
# Optional persistent default for new terminal sessions:
[Environment]::SetEnvironmentVariable("FAUSTUS_DATABASE",
"D:\Faustus\faustus.sqlite3", "User")
```

The development launcher reads `FAUSTUS_DATABASE` from the process environment or the Windows user setting, so an already-open editor can use the persisted default too. Explicit `-Database` / `--database` arguments override that default. CLI commands launched from an existing terminal need `$env:FAUSTUS_DATABASE` set in that terminal or a new terminal session.

The optional `--economy-database` / launcher `-EconomyDatabase` (`FAUSTUS_ECONOMY_DATABASE`) override remains available for deliberately split deployments, but is unnecessary for the single-database setup. A separate economy store uses SQLite WAL with NORMAL synchronous mode and bounded per-connection caches for bulk imports: it remains crash-consistent, but an OS/power failure can lose the newest uncheckpointed observations, which can be recollected or reimported. A unified store uses the main database's unchanged durability settings. Allow space for the database, WAL, archive downloads, SQLite temporary files, and build artifacts. Parent directories for a custom database must already exist.

An upgrade of an already populated economy store may rebuild indexes on its first startup. Start the service separately and wait for its listening message before opening the client; do not interrupt that migration. Later starts reuse the persisted indexes and category catalog.

Published dumps provide daily stash estimates, not 15-minute backfill, exchange volume, or listing/sample counts. Only Chaos-quoted currency rows are imported; differently directed quotes are explicitly counted as exclusions, not silently inverted or merged. Unknown historical categories remain browsable under Historical. Original link ranges such as `1-4 links` are kept as ranges, not treated as an exact four-link price. Dumps do not become Gem flipping snapshots: their confidence/variant coverage cannot reconstruct a complete same-instant gem/material bundle.

Attribution: historical data is provided by poe.ninja under the **CC BY-SA** license stated on its data page. Import records retain source URLs, checksums, license attribution, coverage, counts and exclusions. Retain the source attribution and applicable share-alike terms when redistributing imported data. Faustus's own source-code license is unchanged.

The Gem flipping page compares poe.ninja SkillGem variants by level, quality, and corruption. Its shared search filters all three strategies, and clicking a gem row carries that gem into the search when switching strategies. The **As of** selector defaults to **Latest**, followed by locally saved instants, newest first. Buy, sell, confidence counts, and automatic material prices all come from the selected snapshot, not different sparkline endpoints; manual material overrides are optional. Each snapshot preserves the latest known gem and currency overviews together, including their separate fetch times; the feeds are not necessarily updated simultaneously. The Current prices tab always uses current data, even when viewing a saved gem instant. Changing leagues resets the instant to Latest; refreshing the same league preserves a pinned instant.

The strategies estimate level-1-to-max flips, 20% quality upgrades using 20 Gemcutter's Prisms, and approximate expected Vaal corruption returns. Facetor's Lens is an optional, manually entered Chaos cost in Level Flip, including when viewing a saved instant. Its price defaults to 50 Chaos, with one Lens per flip and no quantity control. **Min** in the compact confidence controls configures the sample/listing threshold (default 10, a positive whole number): buy and all valued sell variants must have at least that many samples and listings. Missing counts are low confidence. Low-confidence gems are excluded by default; **Low conf** includes them with a LOW CONF badge. Invalid thresholds show an error instead of silently using another value. Lens XP varies and is not included in the gem feed. Estimates exclude trade fees, leveling time, and market slippage.

Vaal corrupt uses an explicitly simplified four-bucket model on a max-level, 20%-quality uncorrupted gem: +1 level (12.5%), +3 quality (12.5%), max-level Q20 corrupted (62.5%), and max-level Q0-19 corrupted (12.5%). The -1 level outcome is valued after releveling to max. This approximation groups 21-22% quality with +3 quality and treats Vaal conversion as unchanged; it is not an exact mechanics simulation, including for special gems. +1 level and unchanged use their exact corrupted variants. +3 quality prefers Q23, falling back to the lowest listed Q21-23 price; reduced quality uses the lowest listed Q0-19 price. Bucket estimates are flagged in the breakdown. An entirely unpriced bucket excludes the gem; uncorrupted prices and zero valuations are never substituted. The Sell column becomes **EV** (expected value) on this tab: the probability-weighted resale value. Net subtracts the buy price and one Vaal Orb; ROI divides Net by that total cost. Hover or focus EV, Net, or ROI for a table of outcome probabilities, resale valuations, weighted contributions, and summed EV, followed by cost, Net, and ROI formulas and the model assumptions. Other strategies retain the Sell label. Listings shows buy listings / minimum listings across the valued outcomes, with confidence checked on every valued variant.

The breakdown table includes the listing count for each outcome's valued variant, including estimated bucket representatives; missing counts display N/A rather than zero. EV, Net, and ROI expose these details on hover or focus without a separate explanatory label above the table.

The Vaal tab also includes a sortable **Max value** column: the highest resale valuation among the modeled outcomes, before costs and without probability weighting.

GCP and Vaal Chaos prices are editable on the gem page. Empty fields use the selected instant's feed prices, shown as placeholders; typing a value overrides that price immediately. Manual overrides accept

non-negative finite prices, including zero, and persist across strategies, snapshots, and refreshes within a league. Clear a field to restore its feed price; changing leagues resets both overrides. Manual prices do not change saved snapshots or the Current prices tab. Invalid values show an error instead of silently falling back; explicit overrides also work when material feed prices are unavailable.

The service also exposes:

- `GET /health`
- `GET /api/v1/leagues` (cached poe.ninja economy league options)
- `POST /api/v1/selection` with `{"id":"<league-id>"}`
- `GET /api/v1/status`
- `GET /api/v1/economy/categories` (supported live categories plus imported historical-only categories)
- `GET /api/v1/economy/leagues` (live leagues and imported league availability)
- `GET /api/v1/economy/overview?league=LEAGUE_ID&category=Currency&source=exchange` (or `source=stash`, optionally `date=YYYY-MM-DD`)
- `POST /api/v1/economy/refresh?league=LEAGUE_ID&category=Currency&source=exchange` (provider-cache-aware revalidation)
- `GET /api/v1/economy/dates?league=LEAGUE_ID&category=Currency&source=stash`
- `GET /api/v1/economy/history?league=LEAGUE_ID&category=SkillGem&source=stash&item=ITEM_ID` (optionally `days=7`, `30`, etc.; up to 20,000 newest observations)
- `GET /api/v1/economy/archives` (cached published catalog and persisted import status; `refresh=true` rediscovers availability)
- `POST /api/v1/economy/import` with `{"name":"Ancestors"}` (queues an explicit softcore-only import)
- `GET /api/v1/exchange/prices?league=LEAGUE_ID&type=Currency` (also supports `Fragment`; responses are cached for 15 minutes and revalidated with ETags)
- `GET /api/v1/gems?league=LEAGUE_ID` (poe.ninja SkillGem variants, prices, listing counts, and sparkline history; cached for five minutes)
- `GET /api/v1/gem-analysis?league=LEAGUE_ID` (latest gem/material data, explicit independent provider errors, and saved snapshot options; records a snapshot when gem data is available)
- `POST /api/v1/gem-analysis/refresh?league=LEAGUE_ID` (revalidates both feeds with ETags and saves the resulting bundle; concurrent/repeated requests for a league reuse the last refresh attempt for one minute, including explicit provider errors)
- `GET /api/v1/gem-analysis?league=LEAGUE_ID&snapshot=SNAPSHOT_ID` (immutable saved gem/material data for that league; database-only, without contacting providers; missing or cross-league IDs return 404)
- `GET /api/v1/exchange/markets?league=LEAGUE_NAME&limit=100`
- `GET /api/v1/analysis/period?league=LEAGUE_NAME` (known bounds, their provenance, and actual saved daily/hourly observation ticks)
- `GET /api/v1/analysis/flips?league=LEAGUE_NAME&start=UTC_HOUR&end=UTC_HOUR&offset=0&limit=100` (optional `min_price`, `max_price`, GGG-only `min_volume`, `search`, `category`, `venue=exchange|trade`, and `sort=percent-desc|percent-asc|value-desc|value-asc`; returns the matching `total`, page metadata and bounded exclusion diagnostics)

- `POST /api/v1/analysis/sparklines` with `{league,start,end,rows}` (the displayed flip rows only, at most 100; returns `series` with each row `id`, at most 60 `{time,price}` points in historical Chaos, nullable gaps and sampling/coverage warnings)

The market query returns the latest stored snapshot for each pair in each league. The raw ratio and volume fields remain keyed by item identifier; the app does not infer a buy/sell price from them yet.

Build, Test, and Deploy the UI

Run these commands from `crates\market-ui`:

```
npm test                    # Analysis, API transport, and dashboard state
tests
npm run build               # Type-check and create standalone web assets in
dist
npx playwright install chromium   # Once, for browser tests
npm run test:e2e            # Tests the production web build; mocks provider
data
npm run tauri -- build      # Builds desktop assets, executable, and Windows
NSIS installer
```

The web build defaults to same-origin `/api/v1/...` requests. Deploy `dist` as static assets and reverse-proxy `/api` to the market service under the same HTTPS origin. It has no dependency on a Tauri runtime. Set `VITE_API_BASE_URL=https://your-api.example` at build time only when hosting the API on a different origin, and allow the frontend's exact origin on the service:

```
cargo run -p market-service -- serve --allow-origin https://your-
frontend.example
```

`--allow-origin` can be repeated; supplying it replaces the default desktop origins (`tauri://localhost`, `http://tauri.localhost`, and `https://tauri.localhost`). CORS permits GET/POST and JSON Content-Type, without credentials or wildcard origins. Same-origin web requests and Vite's development proxy do not require extra CORS origins.

Desktop packaging uses `.env.desktop` to point directly at its in-process backend at `http://127.0.0.1:8787`. No Tauri command capabilities are granted to the frontend; native startup/shutdown error dialogs are invoked only by Rust. Custom remote deployments should use the web build rather than redirecting the self-contained desktop package. A remote API should use HTTPS. The service is still unauthenticated and its collector selection is global: CORS is not authentication, and a public multi-user deployment needs access control and a decision about collector ownership before exposing selection or ingestion.

The GPUI crate and its tests remain available:

```
cargo run -p market-ui-deprecated --bin faustus-desktop-deprecated
cargo test -p market-ui-deprecated
```

The replacement implements tested TypeScript gem calculations, including same-instant snapshots, configurable confidence filtering, material costs, sorting, the 100-result cap, and 50-row batches. The sortable Type column and shared type filter use one category per gem: Skill, Support, Exceptional, Transfigured, or Trarthus. Special categories come from poe.ninja's `metadata.gemKind`, exposed by the service as `gem_kind`, and take precedence over Skill/Support. Untagged names ending in " Support" are Support; other untagged gems are Skill. Unrecognized provider categories appear as Unknown rather than being guessed from gem names. Type, search, and confidence settings persist across strategies, instants, and refreshes and are applied before the 100-result cap. Tables use native browser scrolling, including middle-button autoscroll where supported by the browser/platform. Search persists across gem strategies; clicking a gem name selects that name as the search.

The Type dropdown is a multi-select list: click a type to toggle its highlight, without checkboxes. The exclusive **All** option is selected by default, while individual types are unselected. Choosing a type deselects All; choosing All clears individual selections and includes every category (including Unknown). With individual types selected, a gem matches any of them. Deselecting every individual type shows no opportunities. Its fixed-width summary shows All, None, one selected name, or a count; the tooltip retains the full selection. Selections persist across strategies, snapshots, and refreshes. The dropdown supports keyboard navigation, Escape to close, and clicking or tabbing outside to dismiss. The date dropdown uses compact Canadian date labels such as `Nov 9, 19:05` in the browser's local timezone, with full local dates and timezone names in option tooltips. Stored timestamps and snapshot ordering remain unchanged.

All UI dates use Canadian English (`en-CA`) formatting and the browser's local timezone. Full timestamps use year-month-day dates, 24-hour time, and a timezone name; snapshot options use an abbreviated month. Latest includes the latest gem overview's fetch time (for example, `Latest 19:56`), with its full local timestamp in the tooltip, even when viewing a saved instant. The gem page omits strategy descriptions and the data/explanation caption; calculation caveats remain documented above. API and database timestamps remain unchanged.

The Aurelia application uses the full `@aurelia/router` package, not router-lite. Current prices, Currency Exchange, Flip analysis, Triangular arbitrage, Gem flipping and Settings are real routes at `#/prices`, `#/exchange`, `#/flips`, `#/triangular-arbitrage`, `#/gems` and `#/settings`, rendered in an `au-viewport`; the empty URL redirects to prices and the old `#/arbitrage` route remains an alias. Hash routing works in both the web frontend and the Tauri shell without server-side route rewrites. Direct links, reload and browser back/forward work.

`MarketApi` and `MarketSession` are registered as container-scoped singletons and obtained with `resolve(...)`. Each routed page owns its behavior directly in its matching TypeScript class; Aurelia's Vite conventions pair it with the matching HTML template without manual `CustomElement.define` calls. The session owns shared live-league selection, imported datasets and the historical day. It also retains explicit data-only page snapshots so filters, saved snapshot selection, item details and chart periods survive route-controller destruction within the running app, but not browser reloads. Active page references let the shared header delegate date selection and contextual refresh; those references are cleared when pages

unload. Page requests and dump polling stop on unload, and DOM refs and request controllers are never saved in page snapshots. Tests can override service registrations through a DI container.

Frontend source organization:

- Each active component has a same-name `.ts` view-model and `.html` template: `src/pages/current-prices/current-prices`, `src/pages/gem-flipping/gem-flipping`, `src/pages/settings/settings-page` and `src/pages/settings/dump-manager`. The dump manager owns dump discovery, imports, status polling and request cleanup. Obsolete price/GGG template-only pages have been replaced or removed.
- `src/core/` contains the HTTP client, API types, gem-analysis calculations, and their unit tests.
- `src/services/` contains the injectable shared market session; page behavior and its tests live alongside each page's template.
- `src/` retains the entry point, environment declarations, shared header/router shell and shared styles. The shell coordinates shared selectors and contextual refresh, not page calculations or chart logic.

Data Collection Notes

Normal service operation does not start the old GGG global historical cursor collector. Explicit Currency Exchange range imports use the compact hourly store described above; the old records and implementation remain available for deprecated integrations without deleting or rewriting their data.

poe.ninja's supported economy API supplies category overviews, not a supported full item-detail history API. The service records applicable categories for the selected live league on a 15-minute cycle with staggered requests. Provider responses are shared by economy and gem/material consumers, cached by league/category/source and conditionally revalidated with ETags. Manual refresh respects provider cache lifetimes; 304 responses retain original source timestamps instead of manufacturing new observations. Economy API requests go through the backend, not from end-user browsers. Item images load directly from `https://web.poecdn.com`; the desktop CSP permits that image origin only, without expanding script or API access. Send an appropriate descriptive User-Agent using `--user-agent`.

Gem-analysis snapshots are recorded in SQLite when the dashboard loads or requests Latest. An independent server collector also revalidates the selected league's gem and material feeds immediately after league selection and every 15 minutes thereafter, without needing an open client or waiting for the GGG exchange backlog. Collection switches with the selected league and continues while the service runs; after a service restart it waits for a client to select a league. Failures are logged and retried on the next interval. There is no historical backfill.

The client's **Refresh** button requests cache-aware revalidation; a shared provider response stays cached until its cache lifetime allows another conditional request. Gem snapshot capture also has a one-minute per-league cooldown, including failed attempts, and serializes captures to prevent duplicate refreshes. ETags are retained; an unchanged provider response keeps its original source timestamp and identical bundles reuse their existing snapshot rather than manufacturing a new instant. A pinned historical instant stays selected on Refresh, while Current prices and the latest snapshot are updated separately.

The selector lists the 200 most recent saved instants (plus a pinned older instant, if needed); older records remain stored. Snapshots survive service restarts. Previously collected Currency/Fragment stash history

cannot reconstruct historical gem variants. The documented poe.ninja API only exposes current overviews, and its league list supplies no league-start timestamps, so saved instants use actual capture timestamps displayed in local time rather than invented Hour/Day/Week labels. If material prices failed at capture time, that failure is preserved and material-dependent strategies stay unavailable unless a manual material price is entered; current material prices are never substituted automatically. Restart the market service after updating to enable periodic gem collection and the explicit refresh endpoint.

Only the deprecated GPUI WASM frontend needs nightly because GPUI Kit's web backend enables `wasm_thread`. The Aurelia web frontend uses the standard Vite build and does not compile Rust to WebAssembly.

GGG's API and poe.ninja's economy API can change without notice. Respect their rate limits and caching guidance. poe.ninja permits use of its documented economy endpoints but prohibits directly replicating its site; this app is intended for cross-source market analysis. Do not use undocumented site endpoints.

This product isn't affiliated with or endorsed by Grinding Gear Games in any way.

Workspace

- `market-core`: shared identifiers, exact decimal quotes, comparisons, and API types.
- `faustus-fetch`: GGG hourly/cursor-page HTTP client.
- `ninja-fetch`: documented economy client, ETag handling, and overview DTOs.
- `market-store`: SQLite migrations, compact hourly market/coverage storage, legacy cursor recording, economy/import history, and gem snapshots.
- `market-service`: local economy and analysis HTTP APIs, resumable hourly imports, scheduled poe.ninja collectors, and ingestion/import CLI.
- `market-ui`: Aurelia v2/Vite frontend in `src`, with the Tauri v2 Rust desktop shell in `src-tauri`.
- `market-ui-deprecated`: preserved GPUI desktop and single-threaded WebAssembly frontends; major pages live under `src/pages/`.

The deprecated frontends use GPUI Kit 0.9.1, which pins the matching GPUI 0.3.12 snapshot and platform crates. Rust 1.92 or newer is required by the current dependency graph.